Programación Orientada a Objetos
Boletín 1

El objetivo de este boletín es introducir los conceptos de clase, objeto y encapsulación. Más concretamente, se trabajarán los siguientes aspectos:

- Estructura de una clase: atributos, setters/getters, métodos y constructores.
- Modificadores de acceso a los elementos de una clase.
- Mecanismo de **aliasing**.
- Mecanismo de **autoboxing/unboxing**.
- Método **toString**.
- Método **equals**.

Para ello, en los ejercicios de este boletín se crearán las diferentes clases que representarán las características de un producto; un paquete, que podría estar formado por varios productos; y una empresa de mensajería, que se encarga de gestionar el envío de los paquetes.

Ejercicio 1

1. Crear un proyecto denominado B1E1 en el que se crearán dos clases: una clase Producto que representa las características principales de un producto y otra que será la clase PrincipalB1E1 en la que se crearán instancias/objetos de la clase Producto para testear que los métodos y constructores están correctamente implementados.

2. Crear la clase que representa las características de un producto.

Nombre de la clase: Producto

Atributos:

- numSerie (String): número de serie del producto.
- nombre (String): nombre del producto.
- descripcion (String): descripción del producto.
- marca (String): marca del producto.
- peso (Float): peso del paquete.
- precio (Float): precio del paquete.

Constructores:

Se deben implementar los siguientes constructores, en los que se deben cumplir las restricciones de valores indicadas para los setters de los atributos:

- Constructor con los siguientes argumentos: numSerie (String).
- Constructor con los siguientes argumentos: numSerie (String), nombre (String), peso (float) y precio (float).

Getters/Setters:

Todos los atributos de la clase deben tener los correspondientes getters implementados.

Los atributos nombre, descripcion, marca, peso y precio deberán tener los setters implementados con las siguientes restricciones:

- nombre puede tomar cualquier valor, pero tendrá un máximo de 30 caracteres, truncando el texto cuando se alcance ese tamaño.
- descripcion puede tomar cualquier valor, pero tendrá un máximo de 200 caracteres, truncando el texto cuando se alcance ese tamaño.
- marca debe tomar uno de los valores de las siguientes marcas: "HP", "Amazon", "Microsoft", "iPhone", "Samsung" y "Asus".
- peso debe ser mayor que 0.
- precio debe ser mayor que 0.

Métodos funcionales:

- public String toString(): la representación en cadena de texto que debe devolver seguirá la siguiente forma:

```
{
  "numSerie": "<valor>",
  "nombre": "<valor>",
  "descripcion": "<valor>",
  "marca": "<valor>",
  "peso": "<valor>",
  "precio": "<valor>"
}
```

  donde <valor> es el valor del atributo correspondiente. Además, si los atributos no tienen valores (es decir, son null), entonces no formarán parte de la representación en cadena de texto del objeto correspondiente.

3. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: PrincipalB1E1

Operaciones a realizar en la función main():

a. Crear un producto cuyo numSerie es "A0001B".

b. Mostrar por pantalla la representación en cadena de texto del producto creado en (**a**).

c. Escribir los siguientes valores para los atributos del producto creado en (**a**):

- nombre: "Echo Dot".
- descripcion: "Te presentamos el Echo Dot: nuestro altavoz inteligente más popular. Viene con un diseño con acabado en tela que se ajusta perfectamente a cualquier espacio pequeño. Controla la música con la voz: reproduce en streaming canciones en Amazon Music, Spotify, TuneIn y otros servicios. Un sonido más intenso y de mayor calidad: vincula un segundo Echo Dot (3.ª generación) para obtener un sonido estéreo más profundo. Si tienes dispositivos Echo compatibles repartidos en varias habitaciones, podrás llenar de música toda tu casa. Lista para ayudarte: pídele a Alexa que reproduzca música, responda a preguntas, narre las noticias, consulte la previsión del tiempo, configure alarmas, controle dispositivos de Hogar digital compatibles y mucho más.".
- marca: "Amazon".
- precio: 33,99.
- peso: 0,30.

d. Mostrar por pantalla los valores de cada uno de los atributos del producto creado en (**a**).

e. Mostrar por pantalla la representación en cadena de texto del producto creado en (**a**).

f. Crear un producto con los siguientes valores para los atributos:

- numSerie: "C0003F".
- nombre: "Impresora LaserJetM110we".
- precio: null.
- peso: 0.

g. Mostrar por pantalla los valores de cada uno de los atributos del producto creado en (**f**).

h. Escribir los siguientes valores para los atributos del producto creado en (**f**):

- descripcion: "Impresión a doble cara de forma automática, Wi-Fi de doble banda con restablecimiento automático, Wi-Fi Direct, Fast Ethernet, HP Smart App, USB 2.0 de alta velocidad, Apple AirPrint y Mopria.".
- marca: "HP".

i. Escribir un valor correcto para el atributo precio del producto creado en (**f**).

j. Mostrar por pantalla los valores de los atributos descripcion, marca y precio del producto creado en (**f**).

k. Mostrar por pantalla la representación en cadena de texto del producto creado en (**f**).

Ejercicio 2

1. Crear un proyecto denominado B1E2 en el que se crearán dos clases: una clase Paquete que representa las características principales de un paquete y otra que será la clase PrincipalB1E2 en la que se crearán instancias/objetos de la clase Paquete para comprobar que los métodos y constructores han sido correctamente implementados. Además, hay que copiar en el proyecto la clase Producto creada en el Ejercicio 1.

2. Crear la clase que representa las características de un paquete.

Nombre de la clase: Paquete

Atributos:

- id (String): identificador del paquete.
- descripcion (String): descripción del paquete.
- destino (String): población a la que se enviará el paquete.
- trayectoActual (String[]): conjunto de poblaciones por las que ha pasado el paquete.
- productos (Producto[]): productos que forman parte del paquete.
- peso (Float): peso del paquete.
- precio (Float): precio del paquete.

Constructores:

Se deben implementar los siguientes constructores, en los que se deben cumplir las restricciones de valores indicadas para los setters de los atributos:

- Constructor con los siguientes argumentos: id (String).
- Constructor con los siguientes argumentos: id (String), destino (String) y productos (Producto[]).

Getters/Setters:

Todos los atributos de la clase deben tener los correspondientes getters implementados.

Los atributos descripcion, destino, precio y productos deben tener los setters implementados con las siguientes restricciones:

- descripcion puede tomar cualquier valor, pero tendrá un máximo de 200 caracteres, truncando el texto cuando se alcance ese tamaño.
- destino debe tomar uno de los valores de las siguientes poblaciones: "A Coruña", "Santiago de Compostela", "Lugo", "Vigo", "Pontevedra", "Ourense", "Lalín", "Verín", "Muros", "Noia" o "Ferrol".
- setProductos, además de escribir el valor de los productos, debe actualizar automáticamente el valor del peso del paquete, invocando para ello al método calcularPeso.
- precio debe ser mayor que el valor obtenido por el método calcularPrecioMinimo. Cabe destacar que si no se dispone de los productos del paquete no se podrá escribir el valor para el atributo precio.

Métodos funcionales:

- public boolean actualizarTrayectoria(String poblacion): la trayectoria del paquete se actualiza con la nueva poblacion, de modo que el valor de la primera posición del atributo trayectoriaActual será la primera población introducida, mientras que el valor de la última posición será el destino del paquete. Por lo tanto, una vez haya sido introducida la población que es el destino del paquete, no se podrán volver a introducir nuevos valores en el atributo. Si se ha actualizado correctamente la trayectoria, se devolverá true; en caso contrario, false.

- private float calcularPeso(): devuelve el peso del paquete como la suma de los pesos de los productos del paquete más un 10% de dicha suma (entendida como el peso del embalaje de los productos).
- private float calcularPrecioMinimo(): devuelve el precio mínimo del paquete, calculado de la siguiente forma:
  a. Si suma_pesos < 1,0 **=>** precio_mínimo = 2,5
  b. Si 1,0 < suma_pesos < 3,0 **=>** precio_mínimo = 4,0
  c. Si suma_pesos > 3,0 **=>** precio_mínimo = suma_pesos * 1,05

  donde suma_pesos es la suma de los pesos de los productos del paquete y precio_mínimo es el valor que debe devolver el método.
- public String toString(): la representación en cadena de texto que debe devolver seguirá la siguiente forma:

```
{
  "id": "valor",
  "descripcion": "<valor>",
  "destino": "<valor>",
  "trayectoriaActual": [ "<valor1>", "<valor2>", ..., "<valorN>" ],
  "productos": [ "<valor1>", "<valor2>", ..., "<valorM>"],
  "peso": "<valor>",
  "precio": "<valor>"
}
```

  donde <valor> es el valor del atributo correspondiente. Además, si los atributos no tienen valores (es decir, son null), entonces no formarán parte de la representación en cadena de texto del objeto correspondiente.

3. Modificar la clase Producto de la siguiente forma:

Atributos:

- paquete (Paquete): instancia del paquete en el que se enviará el producto.

Getters/Setters:

El atributo paquete debe tener los correspondientes getter y setter implementados.

El setter del atributo paquete no tiene ninguna restricción adicional.

4. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: PrincipalB1E2

Operaciones a realizar en la función main():

a. Crear tres productos diferentes e introducirlos en un array.

b. Crear un paquete cuyo id es "20220915AA".

c. Escribir los siguientes valores para los atributos del paquete creado:
- descripcion: "El paquete tiene tres productos que hay que enviar conjuntamente para completar lo que es realmente un producto completo".
- destino: "Noia".
- productos: el array de productos creado en el punto (**a**).

d. Escribir el paquete creado en (**b**) en el atributo paquete de los tres productos creados en (**a**).

e. Mostrar por pantalla los valores de cada uno de los atributos del paquete creado en (**b**).

f. Mostrar por pantalla la representación en cadena de texto del paquete creado en (**b**).

g. Introducir "Lugo" en el atributo trayectoriaActual del paquete creado en (**b**).

h. Introducir "Santiago de Compostela" en el atributo trayectoriaActual del paquete creado en (**b**).

i. Introducir "Lugo" en el atributo trayectoriaActual del paquete creado en (**b**).

j. Introducir "Noia" en el atributo trayectoriaActual del paquete creado en (**b**).

k. Introducir "Vigo" en el atributo trayectoriaActual del paquete creado en (**b**).

l. Mostrar por pantalla la representación en cadena de texto del paquete creado en (**b**).

m. Crear un producto que sea diferente de los productos creados en (**a**), que tenga un precio mayor de 40 y asignar el paquete creado en (**b**) al atributo paquete de dicho producto.

n. Escribir el producto creado en (**m**) en la segunda posición del array creado en el punto (**b**).

o. Mostrar por pantalla el valor del atributo productos del paquete creado en (**b**). ¿Qué es lo que ocurre? Explícalo directamente en el código.

p. Crear tres productos diferentes e introducirlos en un array. Estos productos deben ser diferentes de los creados en (**a**) y en (**m**).

q. Crear un paquete con los siguientes valores para los atributos:

- id: "20220915AB".
- destino: "Vigo".
- productos: el array de productos creado en el punto (**p**).

r. Mostrar por pantalla la representación en cadena de texto del paquete creado en (**q**).

s. Introducir "Vigo" en el atributo trayectoriaActual del paquete creado en (**q**).

t. Mostrar por pantalla el valor del atributo trayectoriaActual del paquete creado en (**q**).

Ejercicio 3

1. Crear un proyecto denominado B1E3 en el que se crearán dos clases: una clase Mensajeria que representa las características principales de una empresa de mensajería y otra clase PrincipalB1E3 en la que se crearán instancias/objetos de la clase Mensajeria para comprobar que los métodos y constructores han sido correctamente implementados. Además, hay que copiar las clases Paquete y Producto del Ejercicio 2.

2. Crear la clase que representa las características de una empresa de mensajería.

Nombre de la clase: Mensajeria

Atributos:

- cif (String): identificador fiscal de la empresa de mensajería.
- nombre (String): nombre de la empresa.
- paquetes (Paquete[]): conjunto de paquetes que han sido enviados por la empresa.
- beneficio (Float): beneficio que ha obtenido la empresa.

Constructores:

Se deben implementar los siguientes constructores, en los que se deben cumplir las restricciones de valores indicadas para los setters de los atributos:

- Constructor sin argumentos.
- Constructor con los siguientes argumentos: cif (String) y nombre (String).

Getters/Setters:

Todos los atributos de la clase deben tener los correspondientes getters implementados.

Los atributos cif y nombre deben tener los setters implementados.

Métodos funcionales:

- public void enviarPaquete(Paquete paquete): se incluirá el paquete en la lista de paquetes enviados por la empresa de mensajería. Se tendrá en cuenta si el paquete ha sido incluido previamente en la lista de paquetes enviados. Además, también se deberá actualizar el beneficio de la empresa invocando al método calcularBeneficio.
- public String[] marcasFrecuentes(): obtener la marca o marcas que han sido enviadas en un mayor número de ocasiones. Se devolverá un array debido a que es posible que existan varias marcas que hayan sido enviadas el mismo número de veces.
- public Producto[] productosCaros(float precioMin): obtener el conjunto de productos cuyo precio es mayor que el precioMin indicado como argumento del método.
- private float calcularBeneficio(): devuelve el beneficio obtenido por la empresa, entendido como la suma del precio de todos los paquetes que han sido enviados.
- public String toString(): la representación en cadena de texto que debe devolver seguirá la siguiente forma: :

```
{
  "cif": "<valor>",
  "nombre": "<valor>",
  "paquetes": [ "<valor1>", "<valor2>", ..., "<valorN>" ],
  "beneficio": "<valor>"
}
```

donde <valor> es el valor del atributo correspondiente. Además, si los atributos no tienen valores (es decir, son null), entonces no formarán parte de la representación en cadena de texto del objeto correspondiente.

3. Modificar la clase Paquete de la siguiente forma:

Atributos:

- mensajeria (Mensajeria): instancia de la empresa que ha enviado el paquete.

Getters/Setters:

El atributo mensajeria debe tener el correspondiente getter y setter implementado.

El setter del atributo mensajeria no tiene ninguna restricción adicional.

Métodos funcionales:

- public boolean equals(Object obj): implementar el método que evalúa la igualdad entre dos paquetes. El criterio de igualdad será el siguiente: dos paquetes son iguales si tienen el mismo id.

4. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: PrincipalB1E3

Operaciones a realizar en la función main():

- **a.** Reutilizar el código correspondiente a los apartados del (**a**) al (**t**) del **Ejercicio 2**.
- **b.** Crear una empresa de mensajería cuyo cif es "AA1234Z" y su nombre es "PRONTO&TARDE".
- **c.** Introducir los paquetes creados en (**a**) en el conjunto de paquetes enviados por la empresa creada en (**b**), invocando al método enviarPaquete.
- **d.** Mostrar por pantalla la representación en cadena de texto de la empresa creada en (**b**).
- **e.** Escribir la empresa creada en (**b**) en el atributo mensajeria de los paquetes creados en (**a**).
- **f.** Crear un paquete cuyo id es "20220915AB".
- **g.** Introducir el paquete creado en (**f**) en el conjunto de paquetes enviados por la empresa creada en (**b**).
- **h.** Mostrar por pantalla la representación en cadena de texto de la empresa creada en (**b**).
- **i.** Obtener las marcas más frecuentes de productos que han sido enviadas por la empresa creada en (**b**).
- **j.** Obtener todos los productos de la empresa creada en (**b**) con un precio mayor de 20.
- **k.** Mostrar por pantalla los valores de cada uno de los atributos del paquete creado en (**b**).

Ejercicios opcionales

A continuación, se proponen una serie de ejercicios opcionales que no serán puntuables para la calificación de la materia, pero que son útiles a la hora de entender los conceptos relacionados con el uso de la clase String, el mecanismo de **autoboxing/unboxing** y las referencias (incluyendo **aliasing**).

Ejercicio opcional 1: Uso de la clase String.

Crear un proyecto denominado B1EO1 en el que se creará únicamente la clase PrincipalB1EO1 para poder realizar operaciones con la clase String y comprobar el uso de referencias; más concretamente, el objetivo del ejercicio es comprobar cómo funciona la asignación de referencias con la clase String y con las diferentes formas de reservar memoria para los objetos de dicha clase.

Operaciones a realizar en la función main():

a. Crear una instancia de la clase String denominada cad1 usando el operador new y asignarle el valor "cadena1".

b. Crear una instancia de la clase String denominada cad2 usando el operador new y asignarle el valor "cadena1".

c. Introducir dos trozos de código en los que se comprueba si cad1 y cad2 tienen el mismo valor usando:

1. El operador ==.
2. El método equals.

En ambos casos, si tienen el mismo valor, se mostrará por pantalla el mensaje "Los objetos son iguales", mientras que si tienen un valor diferente se mostrará el mensaje "Los objetos son diferentes".

d. ¿Los resultados obtenidos en (**c.1**) y (**c.2**) son los mismos? **Explica por qué se obtienen esos resultados**.

e. Crear una instancia de la clase String denominada cad3 y asignarle directamente el valor "cadena1".

f. Crear una instancia de la clase String denominada cad4 y asignarle directamente el valor "cadena1".

g. Introducir dos trozos de código en los que se comprueba si cad3 y cad4 tienen el mismo valor usando:

1. El operador ==.
2. El método equals.

En ambos casos, si tienen el mismo valor, se mostrará por pantalla el mensaje "Los objetos son iguales", mientras que si tienen un valor diferente se mostrará el mensaje "Los objetos son diferentes".

h. ¿Los resultados obtenidos en (**g.1**) y (**g.2**) son los mismos? **Explica por qué se obtienen esos resultados**.

i. Asignar la instancia creada en (**f**) a la instancia creada en (**a**), es decir, cad1 = cad4.

j. Introducir dos trozos de código en los que se comprueba si cad1 y cad4 tienen el mismo valor usando:

1. El operador ==.
2. El método equals.

En ambos casos, si tienen el mismo valor, se mostrará por pantalla el mensaje "Los objetos son iguales", mientras que si tienen un valor diferente se mostrará el mensaje "Los objetos son diferentes".

k. ¿Los resultados obtenidos en (**j.1**) y (**j.2**) son los mismos? **Explica por qué se obtienen esos resultados**.

Ejercicio opcional 2: Uso de los mecanismos unboxing y autoboxing.

Crear un proyecto denominado B1EO2 en el que se creará únicamente la clase PrincipalB1EO2 para poder realizar operaciones con los wrappers de Java a los tipos básicos de datos, comprobando el uso de unboxing y autoboxing así como las referencias entre esos wrappers.

Operaciones a realizar en la función main():

a. Crear una instancia de la clase Integer denominada int1 usando el operador new y asignarle el valor 10.

b. Crear una instancia de la clase Integer denominada int2 usando el operador new y asignarle el valor 10.

c. Introducir dos trozos de código en los que se comprueba si int1 e int2 tienen el mismo valor usando:

1. El operador ==.
2. El método equals.

En ambos casos, si tienen el mismo valor, se mostrará por pantalla el mensaje "Los objetos son iguales", mientras que si tienen un valor diferente se mostrará el mensaje "Los objetos son diferentes".

d. ¿Los resultados obtenidos en (**c.1**) y (**c.2**) son los mismos? **Explica por qué se obtienen esos resultados**.

e. Crear una instancia de la clase Integer denominada int3 y asignarle directamente el valor 10.

f. Introducir dos trozos de código en los que se comprueba si int1 e int3 tienen el mismo valor usando:

1. El operador ==.
2. El método equals.

En ambos casos, si tienen el mismo valor, se mostrará por pantalla el mensaje "Los objetos son iguales", mientras que si tienen un valor diferente se mostrará el mensaje "Los objetos son diferentes".

g. ¿Los resultados obtenidos en (**f.1**) y (**f.2**) son los mismos? **Explica por qué se obtienen esos resultados**.

h. Asignar la instancia creada en (**e**) a la instancia creada en (**b**), es decir, int1 = int3.

i. Asignar 20 a la instancia creada en (**e**), es decir, int3 = 20.

j. Introducir dos trozos de código en los que se comprueba si int1 e int3 tienen el mismo valor usando:

1. El operador ==.
2. El método equals.

En ambos casos, si tienen el mismo valor, se mostrará por pantalla el mensaje "Los objetos son iguales", mientras que si tienen un valor diferente se mostrará el mensaje "Los objetos son diferentes".

k. ¿Los resultados obtenidos en (**j.1**) y (**j.2**) son los mismos? **Explica por qué se obtienen esos resultados**.

l. Mostrar por pantalla los valores de las instancias int1 e int3. **Explica los resultados que se sacan por pantalla**.

Ejercicio opcional 3: Uso de aliasing y del método equals.

Crear un proyecto denominado B1EO3 en el que se creará únicamente la clase PrincipalB1EO3 para poder comprobar el efecto del aliasing y el uso de método equals. Aunque esto ya se ha visto en los ejercicios obligatorios, en este ejercicio se insistirá específicamente en estos conceptos.

Métodos de la clase:

a. Crear el método public static int suma(Integer[] arr), que devuelve la suma de los valores de todos los elementos del array arr.

b. Crear el método public static Integer[] suma10(Integer[] arr), que devuelve un array con el mismo tamaño que el array arr que se pasa como argumento al que se le ha sumado 10 a todas las posiciones. Por ejemplo, si el array que se pasa como argumento es { 10, 20, 30 }, entonces el método devolverá { 20, 30, 40 }.

Operaciones a realizar en la función main():

a. Crear tres instancias de la clase Integer denominadas int1, int2 e int3 y asignarles directamente los valores 10, 20 y 30, respectivamente.

b. Crear un array de Integer de tamaño 3 denominado arr1 y asignar la instancia int1 a la posición 0; int2 a la posición 1; y finalmente, int3 a la posición 2.

c. Introducir un trozo de código en el que se muestre por pantalla el contenido del array creado en (**b**).

d. Crear un array de Integer de tamaño 3 denominado arr2 que se inicializa a los valores 40, 50 y 60.

e. Asignar el array creado en (**d**) al array creado en (**b**), es decir, arr1 = arr2.

f. Modificar el valor de la posición 1 del arr1, asignándole una instancia de Integer creada con new y que tenga un valor de 70.

g. Mostrar por pantalla el contenido de arr1 y arr2. **Explica por qué se obtienen esos resultados**.

h. Modificar el valor de la posición 0 del arr2, asignándole una instancia de Integer creada de forma directa con un valor de 80.

i. Mostrar por pantalla el contenido de arr1 y arr2. **Explica por qué se obtienen esos resultados**.

j. Invocar el método suma con arr1 como argumento y mostrar el resultado por pantalla.

k. Modificar el valor de la posición 2 del arr2, asignándole una instancia de Integer creada de forma directa con un valor de 90.

l. Invocar el método suma con arr1 como argumento y mostrar el resultado por pantalla. ¿El resultado que se obtiene es diferente del resultado de (**j**)? **Explica por qué se obtienen esos resultados**.

m. Invocar el método suma10 con arr2 como argumento y mostrar el resultado por pantalla.

n. Mostrar por pantalla el contenido de arr1 y arr2. **Explica por qué se obtienen esos resultados**.